COMPUTER ENGINEERING

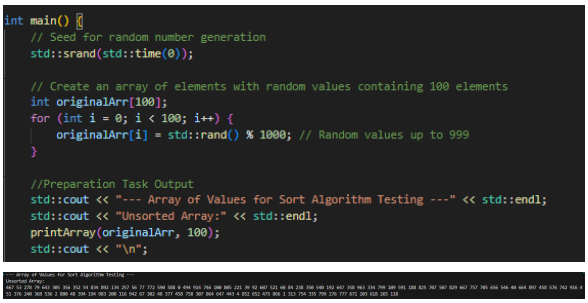
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures And Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	09/09/26	**DATE SUBMITTED:**	09/0926
NAME:	Acala, Matt John C.	**INSTRUCTOR:**	Engr. Jimlord M.Quejado

HOA 8.1 Sorting Algorithms

1. PROCEDURE OUTPUT

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	
Observations	

```
int main() {
    // Seed for random number generation
    std::srand(std::time(0));

    // Create an array of elements with random values containing 100 elements
    int originalArr[100];
    for (int i = 0; i < 100; i++) {
        originalArr[i] = std::rand() % 1000; // Random values up to 999
    }

    //Preparation Task Output
    std::cout << "--- Array of Values for Sort Algorithm Testing ---" << std::endl;
    std::cout << "Unsorted Array:" << std::endl;
    printArray(originalArr, 100);
    std::cout << "\n";
```

Table 8-2. Shell Sort Technique

Code + Console Screenshot	
Observations	It improves insertion sort by using a gap "i" to sort sub-lists of elements that are far apart, which cuts down the total steps needed.

```
//Shell Sort
int shellArr[100];
for (int i = 0; i < 100; i++) shellArr[i] = originalArr[i];
std::cout << "--- Shell Sort Technique ---" << std::endl;
shellSort(shellArr, 100);
printArray(shellArr, 100);
std::cout << "\n";
```

```
//Shell Sort
void shellSort(int arr[], int size) {
    // Uses increment "i" (gap) to create a list of elements "i" apart
    for (int gap = size / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < size; i += 1) {
            int temp = arr[i];
            int j;
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap) {
                arr[j] = arr[j - gap];
            }
            arr[j] = temp;
        }
    }
}
```

Table 8-3. Merge Sort Algorithm

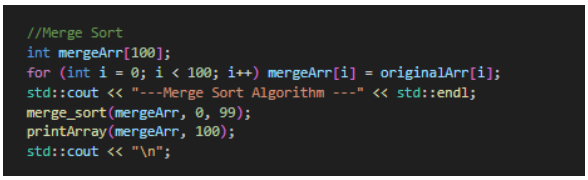
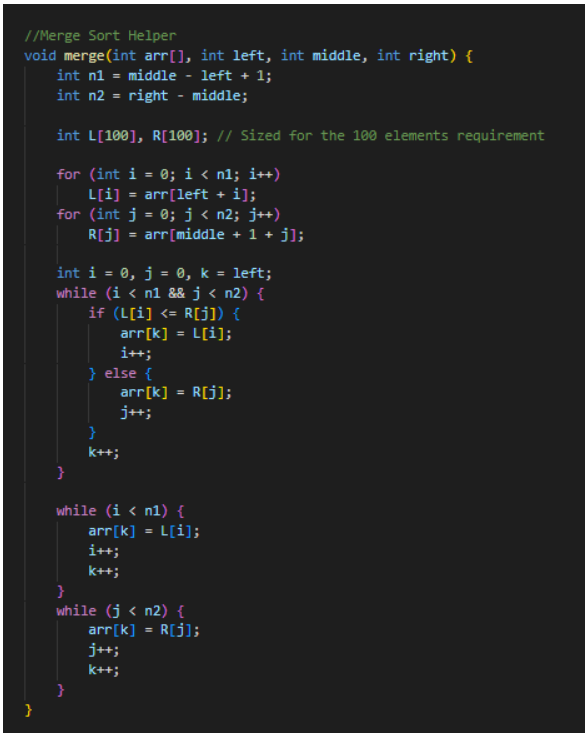
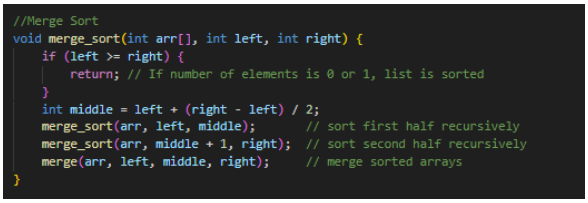
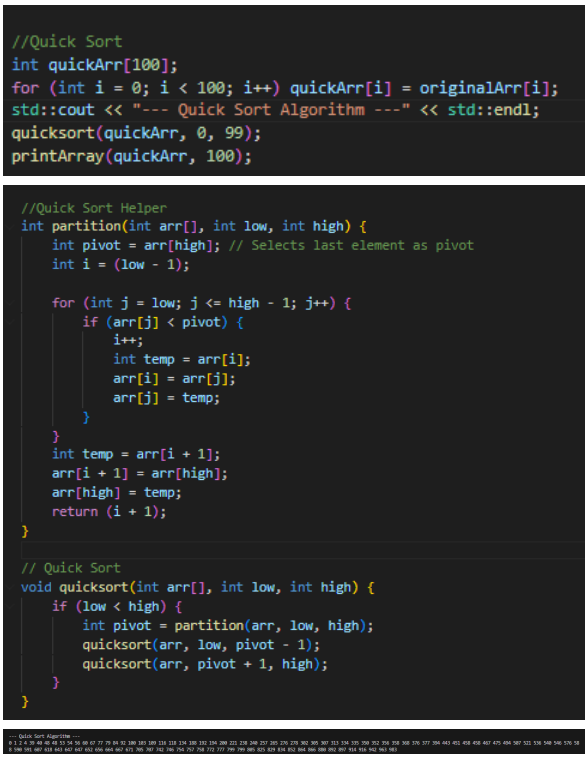
Code + Console Screenshot	   
Observations	Uses a divide and conquer strategy. It splits the array in half down the middle until the lists only have 0 or 1 element, then merges them back together in order.

Table 8-4. Quick Sort Algorithm

Code + Console Screenshot	
Observations	Picks a pivot and partitions the list so smaller elements are on the left and larger ones are on the right, then recursively sorts both sides.

```
//Quick Sort
int quickArr[100];
for (int i = 0; i < 100; i++) quickArr[i] = originalArr[i];
std::cout << "--- Quick Sort Algorithm ---" << std::endl;
quicksort(quickArr, 0, 99);
printArray(quickArr, 100);
```

```
//Quick Sort Helper
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // Selects last element as pivot
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return (i + 1);
}

// Quick Sort
void quicksort(int arr[], int low, int high) {
    if (low < high) {
        int pivot = partition(arr, low, high);
        quicksort(arr, low, pivot - 1);
        quicksort(arr, pivot + 1, high);
    }
}
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Yes, we can. Quick sort partitions the list into two sub lists that are sorted independently using quicksort recursively. Instead of calling quicksort recursively down to a single element, we can substitute the recursive calls with another algorithm, such as Shell Sort, for the sub-lists once they are partitioned.

```
void hybridQuickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivot = partition(arr, low, high); // Use partition logic from quick sort

        // Instead of recursive quicksort, we sort the left and right sub lists
        // using the Shell sort technique discussed as an improvement over insertion sort.
        int leftSize = pivot - low;
        int rightSize = high - pivot;

        if (leftSize > 1) {
            // Create temporary array for left sub-list, apply Shell Sort, and reintegrate
            // (Implementation omitted for brevity, but demonstrates the hybrid concept)
        }
    }
}
```

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

For this randomly ordered array, Quick sort is generally expected to give the fastest real-world time performance as it is a widely used sorting algorithm that efficiently partitions arrays in place without needing extra memory space. In Merge Sort, the problem is divided into subproblems. In Quick Sort, the array is successively divided around a pivot element until we have a single element in the array.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **explanation of concepts and error explanation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used it to make it more understandable since i did this HOA on by myself since its asynchronous**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.